

Evaluation Metrics: Definitions, Mathematics, and Rationale

AI-Powered Repository Intelligence Platform — how each metric in the evaluation harness works, what it measures, and why it belongs in this study

Research question	Does the <i>representation</i> of a repository given to an LLM (raw source vs. dependency graph vs. full knowledge graph) affect the faithfulness and quality of the summary it produces?
Independent factors	Model factor A — qwen2.5-coder:7b, granite-code:3b-instruct, deepseek-coder:6.7b-instruct — and Method / representation factor B — raw, dependency_graph, knowledge_graph. A full factorial design: every model runs under every representation, on every repository.
Dataset	18 repositories (9 Express.js, 9 NestJS), each hand-verified against source
Document scope	Every automatic metric the harness computes, in the order a reader would encounter it in a results table: text overlap → grounded fact-checking → holistic quality → structural accuracy → confidence → statistics

Contents

1. Why one metric is never enough
2. Text-overlap metrics — BLEU-4, ROUGE-L, METEOR, BERTScore
3. Grounded fact-checking — Hallucination score and Coverage score
4. G-Eval — holistic rubric scoring with logprob weighting
5. Structural grounding — the diagram graph-diff F1
6. Calibration — confidence, Platt scaling, Brier score, ECE
7. Efficiency metrics — latency, tokens, cost
8. Statistical rigor — paired significance testing
9. Failure taxonomy
10. Putting it together: reading a results row

1. Why one metric is never enough

A summary that says almost nothing ("*This is a web application.*") can score a perfect 0.0 on a metric that only checks whether stated claims are true — it made no claims to be wrong about. A summary can also share every word with a reference summary and still be useless if a developer can't act on it. No single number captures "quality," so this harness deliberately runs several metrics that fail in different ways, and reports them side by side rather than collapsing them into one score.

Metric family	What it catches	What it misses
Text overlap (§2)	Surface similarity to a reference	A paraphrase that says the same true thing scores low
Grounded fact-checking (§3)	Specific false or missing claims	Fluency, organization, whether it reads well
G-Eval rubric (§4)	Holistic quality a reader would notice	Needs a capable judge model to be reliable
Structural F1 (§5)	Whether the diagram matches real code structure	Says nothing about the prose summary
Calibration (§6)	Whether the model "knows what it doesn't know"	Not a quality score by itself

2. Text-overlap metrics

These compare the model's generated `overview` text against a reference summary (`reference_summaries/<repo>.json`) — the same family of metrics the code-summarization literature this project's lit review cites reports at class/repository level.

2.1 BLEU-4 (Bilingual Evaluation Understudy)

Modified n-gram precision, geometric mean across $n = 1..4$, with a brevity penalty

$$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^4 w_n \log p_n\right)$$

p_n is the precision of n-grams (n consecutive words) in the generated text that also appear in the reference, clipped so a word can't be "matched" more times than it actually occurs in the reference. BP (brevity penalty) is $\frac{1}{\text{gen_len}}$ if the generated text is at least as long as the reference, and $e^{(1 - \text{ref_len} / \text{gen_len})}$ otherwise — a short summary can't game the score by only outputting words it's sure are correct. This project uses $w_n = 0.25$ for each of $n = 1..4$ (the standard "BLEU-4" weighting) with smoothing (method 4), since unsmoothed BLEU-4 is degenerate (exactly 0) whenever a single 4-gram fails to match — near-certain at paragraph length.

Why it's here

Cheap, deterministic, and the most widely reported metric in the cited literature — a number every reviewer already knows how to read, even though (see below) it's a blunt instrument for prose.

Limitation

Purely lexical: a summary that says "creates, reads, updates, and deletes tutorial records" scores near-zero against a reference that says "provides full CRUD for tutorials," despite being synonymous. This is exactly why BLEU never runs alone here.

2.2 ROUGE-L (Longest Common Subsequence)

F-measure over the longest common subsequence (LCS) of tokens

$$R_{\text{lcs}} = \text{LCS}(\text{gen}, \text{ref}) / \text{len}(\text{ref}) \quad P_{\text{lcs}} = \text{LCS}(\text{gen}, \text{ref}) / \text{len}(\text{gen})$$

$$F_{\text{lcs}} = \frac{(1 + \beta^2) R_{\text{lcs}} P_{\text{lcs}}}{R_{\text{lcs}} + \beta^2 P_{\text{lcs}}}$$

Unlike BLEU's fixed n-grams, the longest common subsequence doesn't require matched words to be contiguous — it rewards preserving the reference's overall word *order* without demanding an exact phrase match. This project reports the balanced F-measure ($\beta=1$, i.e. the harmonic mean of precision and recall) rather than precision or recall alone.

Why it's here

More forgiving of legitimate rewording than BLEU while still being purely lexical and deterministic — a useful second lexical signal, not a replacement for BLEU.

2.3 METEOR

Harmonic mean weighted toward recall, with a fragmentation penalty

$$F_{\text{mean}} = \frac{10 \cdot P \cdot R}{R + 9P} \quad \text{METEOR} = F_{\text{mean}} \times (1 - \text{Penalty})$$

P and R are unigram precision and recall (weighted 9:1 toward recall — METEOR was designed on the premise that missing content hurts a summary more than a little extra wording). The fragmentation penalty grows with how many discontinuous "chunks" the matched words fall into, so a generated summary that matches the reference's words but scrambles their order is penalized even though a bag-of-words match would look perfect.

Why it's here

Its recall-weighting is a good match for this project's `coverage_score` spirit (§3.2) applied at the text-overlap level: missing a key fact should cost more than adding a little extra detail.

2.4 BERTScore

Cosine similarity between contextual embeddings, greedily matched, then F1'd

$$R_{\text{BERT}} = \frac{1}{|\text{ref}|} \sum_{x \in \text{ref}} \max_{\hat{x} \in \text{gen}} x^{\vec{o}} \cdot \hat{x}^{\vec{o}} \quad P_{\text{BERT}} = \frac{1}{|\text{gen}|} \sum_{\hat{x} \in \text{gen}} \max_{x \in \text{ref}} x^{\vec{o}} \cdot \hat{x}^{\vec{o}}$$
$$F_{\text{BERT}} = 2 \cdot P_{\text{BERT}} R_{\text{BERT}} / (P_{\text{BERT}} + R_{\text{BERT}})$$

Each word in the generated and reference text is embedded (this project uses `distilbert-base-uncased` — a deliberate speed/accuracy tradeoff against the paper-default `roberta-large`, since the full battery already runs a lot of local inference; see §7). Every reference word is greedily matched to its most similar generated word by cosine similarity, and vice versa, then combined into an F-score — the same shape as BLEU/ROUGE, but comparing *meaning* (embedding space) rather than exact tokens.

Why it's here

This is what fixes BLEU/ROUGE's blind spot: "creates, reads, updates, and deletes tutorial records" and "provides full CRUD for tutorials" embed close together even though they share almost no words, so BERTScore correctly rewards the paraphrase.

Scope decision

Two further metrics the cited literature reports — BLEURT and SIDE — are *not* implemented. BLEURT needs a TensorFlow checkpoint that's awkward to maintain locally; SIDE needs a specific released contrastive-learning checkpoint from its own paper's repository. Neither reproduces from a plain `pip install` the way the four metrics above do. Disclosed here, not silently dropped.

3. Grounded fact-checking

The two metrics in this section don't compare against a human-written reference at all — they compare the summary against *the parser's own static-analysis output* for that exact repository. This is the axis the core research question is actually about: does giving the model more structure reduce how often it states something the code doesn't support?

3.1 Hallucination score (precision-like)

Fraction of the summary's claims that the parser's facts do NOT support

$$\text{hallucination_score} = \frac{\text{unsupported_claims}}{\text{total_claims}} \in [0, 1] \quad (\text{lower is better})$$

A judge model — deliberately a **fourth** model, never one of the three generators being compared (§4.3 of the report explains why self-judging is unsound) — is given the parser's ground-truth facts (detected framework, dependencies, endpoints, entity counts) and the generated summary, and is instructed to flag any specific, checkable claim the facts don't support (a named technology, library, database, or API style that never appears in the facts). Vague claims ("this is a web application") are explicitly excluded by the judge prompt — only specific, falsifiable claims count.

Why it's here

This is the metric the paper's headline result is built on. Ground truth is *static analysis*, not a human's opinion — so it measures faithfulness to what the code actually contains, exactly the property structured context is hypothesized to improve.

3.2 Coverage score (recall-like)

Fraction of the parser's known facts that the summary actually mentions

$$\text{coverage_score} = \frac{\text{total_facts} - \text{missing_facts}}{\text{total_facts}} \in [0, 1] \quad (\text{higher is better})$$

Coverage is hallucination's necessary complement. A summary that says almost nothing can score a perfect 0.0 hallucination (it made no false claims — it made almost no claims). Coverage's fact list is built *deterministically* from the parser's output before the judge ever runs (framework, every dependency, every endpoint, every class/service, every database entity) — so unlike hallucination's claim list, which the judge extracts and could inflate or deflate, coverage's denominator is never at the judge's mercy. Only "is this specific known fact mentioned" needs judgment.

Why it's here

Without coverage, a model could win the ablation by saying less under raw context and more under knowledge_graph — indistinguishable from actually being more faithful. Reporting both side by side rules that out: a genuine improvement should raise coverage *or hold it steady* while lowering hallucination, not trade one for the other.

4. G-Eval — holistic rubric scoring

Hallucination and coverage check individual claims; neither asks "does this read as a good summary." G-Eval (Liu et al., 2023) is the protocol this project uses to answer that — not "ask an LLM for a number 1-5," which is exactly the vague scoring this document exists to rule out, but a specific, published, human-validated procedure.

4.1 The protocol

1. **Fixed chain-of-thought steps per criterion.** Each of the five criteria (Completeness, Conciseness, Correctness, Cohesiveness, Domain Specificity — the exact rubric arXiv:2501.07857 uses) has hand-authored, numbered evaluation steps the judge is walked through, not just a one-line definition. This project fixes these steps once rather than having the judge invent them per run, because a reproducible study needs *a fixed rubric* across every comparison — a self-regenerated rubric would itself be a source of run-to-run noise.
2. **Probability-weighted scoring, not a single sample.** The formula below.

Score = expectation over the score distribution implied by the judge's own token logprobs

$$\text{Score} = \sum_{i=1}^5 P(s = i) \cdot i \quad \text{where} \quad P(s = i) = \frac{e^{\text{logprob}_i}}{\sum_{j=1}^5 e^{\text{logprob}_j}}$$

Instead of trusting whichever digit (1-5) the model happened to sample, the judge's raw API call returns the log-probability it assigned to *every* candidate digit at that position (via Ollama's `logprobs / top_logprobs`). Renormalizing those into a probability distribution (softmax) and taking its expectation produces a continuous score — e.g. 3.8 instead of a coin-flip between "4" and "3" — which is both more discriminating between close cases and less sensitive to sampling noise. When a model/Ollama version doesn't return logprobs, the implementation falls back to sampling `k` times and averaging the parsed integers — still more rigorous than one unweighted sample, and every result records which method (`logprob_weighted` vs. `sample_averaged`) actually produced it, so this is auditable, not silent.

4.2 Criteria used

Target	Criteria	Source
Summary	Completeness, Conciseness, Correctness, Cohesiveness, Domain Specificity	arXiv:2501.07857 (exact rubric)
Diagram	Value, Comprehensibility	CIAO (arXiv:2604.08293) RQ1/RQ2 wording

Deliberately excluded: diagram "accuracy"

CIAO's third dimension (RQ3, "accuracy w.r.t. source code") is *not* re-judged subjectively here. This project's diagrams are generated deterministically from the same Neo4j graph the parser wrote, and §5's structural F1 already checks accuracy against hand-verified ground truth — an objective measurement a judge's guess cannot improve on.

5. Structural grounding — the diagram graph-diff F1

The Mermaid architecture diagram is generated deterministically (no LLM call) from the same Neo4j graph the parser built. "Diagram correctness" therefore reduces to a question about the *parser and graph builder*, not about model behavior — and it's answerable exactly, against a human-verified answer key.

Standard precision / recall / F1, computed separately for modules, imports, and endpoints

$$\text{Precision} = \frac{|\text{Extracted} \cap \text{Expected}|}{|\text{Extracted}|} \quad \text{Recall} = \frac{|\text{Extracted} \cap \text{Expected}|}{|\text{Expected}|} \quad \text{F1} = \frac{2 \cdot \text{P} \cdot \text{R}}{\text{P} + \text{R}}$$

"Extracted" is what the pipeline's parser actually found for a repository (module paths, resolved import edges, endpoint method+path strings); "Expected" is a human-corrected annotation file, seeded from the parser's own output and then verified against the real source — never scored against an uncorrected seed, which would trivially score 1.0 by construction (the pipeline agreeing with itself).

Why import is sometimes reported as recall only

Precision is only interpretable when the annotation is *exhaustive*. Two of the 18 repositories are annotated completely (every real edge), so their import precision is meaningful. For repositories with hundreds of import edges, exhaustive annotation isn't practical — a verified subset is annotated instead, and only recall is reported for those, since precision against an incomplete list would just measure how few edges were annotated, not parser error.

6. Calibration — does the model know what it doesn't know?

A model that is right 80% of the time is more useful if it also *signals* lower confidence on the 20% it gets wrong. This section implements the method arXiv:2404.19318 (Calibration of LLMs on Code Summarization) uses to check that, without needing a reference at generation time.

6.1 Raw confidence

Geometric mean of a generation's own output-token probabilities

$$\text{confidence} = \exp\left(\frac{1}{n} \sum_{t=1}^n \log \text{prob}_t\right)$$

The geometric mean (equivalently, the exponential of the mean log-probability) is used rather than the arithmetic mean or the sum, because — per the cited paper — "the product, not the sum, of individual probabilities better represents the probability of a sequence": a single very-uncertain token should pull the whole sequence's confidence down multiplicatively, the way one weak link weakens a chain, not just nudge an average.

6.2 Turning raw confidence into a validated probability

Platt scaling: logistic regression mapping raw confidence to $P(\text{correct})$

$$P(\text{correct}) = \frac{1}{1 + e^{(a \cdot \text{confidence} + b)}}$$

Raw confidence from an LLM is rarely a well-calibrated probability on its own — models are frequently over- or under-confident in a consistent, correctable direction. (a, b) are fit by maximum likelihood on a batch of (confidence, correct) pairs, where "correct" is defined as the generation's BERTScore-vs-reference clearing a threshold (0.5 by default) — reusing §2.4's metric as the calibration target, rather than inventing a separate correctness definition.

6.3 Validating the calibration itself

Brier score — mean squared error between calibrated probability and outcome

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i)^2$$

Expected Calibration Error — bin by predicted confidence, compare to actual accuracy

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} \cdot |\text{acc}(B_m) - \text{conf}(B_m)|$$

Brier score is 0 for a perfect forecaster and 0.25 for a coin-flip forecaster on a balanced set — lower is better. Brier Skill Score (also reported) normalizes this against the "always predict the base rate" baseline, so a value above 0 means the calibrated confidence is actually informative, not just accurate by luck of the base rate. ECE more directly answers "if the model says 80% confident, is it right about 80% of the time" — data is bucketed into M confidence bins and the gap between each bin's average stated confidence and its actual accuracy is averaged, weighted by bin size.

Scope

This needs a *second* generation call per sample with logprobs requested — real compute cost, not free like the other metrics. Scoped to a representative subset of repositories rather than the full $18 \times 3 \times 3$ battery.

7. Efficiency metrics

Metric	What it is	Why it matters here
Latency (ms)	Wall-clock time for one generation call	Structured context is hypothesized to be <i>faster</i> too — fewer, denser tokens vs. raw source's noise
Input / output tokens	From Ollama's <code>prompt_eval_count</code> / <code>eval_count</code>	Token efficiency (output/input ratio) shows how much summary a representation buys per token spent
Estimated cost (USD)	Always \$0	All three models run locally via Ollama — no paid API calls anywhere in this study

8. Statistical rigor — paired significance testing

Between-repository variance on these metrics is large (standard deviations up to 0.48 on a [0,1]-bounded metric have been observed), so comparing unpaired group means has very low power to detect real differences. The design is **repeated-measures**: the same repository is scored under every (model, representation) combination, so the correct comparison is *paired* — take differences within each (repository, model) pair, which cancels out repository-specific and model-specific difficulty before testing anything.

Mean and 95% confidence interval (t-distribution, appropriate for small per-cell samples)

$$\bar{x} \pm t_{0.975, n-1} \cdot \frac{s}{\sqrt{n}}$$

Wilcoxon signed-rank statistic — sum of signed ranks of the paired differences

$$W = \sum_{i=1}^n \text{sign}(d_i) \cdot R_i \quad \text{where } d_i = a_i - b_i, R_i = \text{rank}(|d_i|)$$

Wilcoxon signed-rank is used rather than a paired t-test because it makes no normality assumption — appropriate for a bounded, often-tied metric at this sample size — and rather than a plain sign test (which the original 6-repository battery used) because it uses the *magnitude* of each paired difference, not just its direction, giving it more statistical power for the same number of observations. Every comparison holds one factor fixed while varying the other (e.g. comparing representations *within* a single model, repeated separately per model) so model and method stay isolated, answerable factors rather than collapsed into one leaderboard column.

9. Failure taxonomy

Aggregate scores hide *what kind* of mistake a model tends to make. Every row is tagged from signals the other metrics already compute — no extra model calls:

Tag	Source signal	Precision
Fabricated endpoint / technology	Hallucination's unsupported claims, pattern-matched	Heuristic — claims are free-form judge-extracted text
Missed endpoint / dependency / framework / class / database entity	Coverage's missing facts (already category-prefixed)	Exact — string-prefix matching on structured data
Over-generic	Overview text length + presence of the repo's actual known terms	Heuristic
Malformed output	Judge or generator output failed to parse	Exact

The precision difference is disclosed deliberately: coverage's structured facts make "missed X" categorization exact string matching, while hallucination's free-text claims make "fabricated X" categorization necessarily approximate. Collapsing fabrication into two broad buckets (endpoint vs. technology) rather than pretending finer-grained precision the underlying signal doesn't have.

10. Putting it together: reading a results row

Every row in the harness's output CSV is one (repository, model, representation) combination, carrying all of the above side by side:

Column group	Answers
Efficiency (§7)	How fast, how many tokens, at what cost
Hallucination / Coverage (§3)	Were the claims true, and was anything important left out
Text overlap (§2)	How close is this to how a human/reference would describe it
G-Eval (§4)	Does it read as a genuinely good summary, holistically
Diagram F1 (§5)	Is the extracted architecture actually right (repo-level, not per-row)
Failure tags (§9)	When it's wrong, what kind of wrong

No single column answers "is knowledge_graph better than raw" on its own — that question is answered by feeding every row's relevant column into §8's paired Wilcoxon test, holding model fixed and comparing representations (and separately, holding representation fixed and comparing models), which is exactly what `app/evaluation/stats.py` automates over the harness's CSV/SQLite output.

Sources: Liu et al. (2023), "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment"; arXiv:2501.07857 (Hierarchical Repository-Level Code Summarization, G-Eval rubric); arXiv:2502.16704 (Code Summarization Beyond Function Level, text-overlap metric suite); arXiv:2604.08293 "CIAO" (architecture-documentation RQ design); arXiv:2404.19318 (Calibration of LLMs on Code Summarization, confidence method). Implementation: `backend/app/evaluation/`.